Object Oriented Programming I

September 18, 2024

Content

When Constructors and Destructors Are Called

Returning a Reference to a private Data Member

Default Memberwise Assignment

const (Constant) Objects and const Member Functions

Composition: Objects as Members of Classes

When Constructors and Destructors Are Called

- Constructors and destructors are called implicitly by the compiler.
- The order in which these function calls occur depends on the order in which execution enters and leaves the scopes where the objects are instantiated.
- Generally, destructor calls are made in the reverse order of the corresponding constructor calls, but the storage classes of objects can alter the order in which destructors are called.

When Constructors and Destructors Are Called

- Constructors are called for objects defined in global scope before any other function (including main) in that file begins execution. The corresponding destructors are called when main terminates.
- The constructor for a local object is called when execution reaches the point where that object is defined. The corresponding destructor is called when execution leaves the object's scope (i.e.,the block in which that object is defined has finished executing).
- Constructors and destructors for local objects are called each time execution enters and leaves the scope of the object.
- The constructor for a static local object is called only once,when execution first reaches the point where the object is defined. The corresponding destructor is called when main terminates. Global and static objects are destroyed in the reverse order of their creation.

CreateAndDestroy Class Definition

```
1  /* ******************************************************
2   * Filename     :   CreateAndDestroy.h
3   * Author       :   Burak Kaleci
4   * Date         :   17.10.2018
5   * Description  :   An example program to introduce
6   *                  constructor and destructor
7   * ******************************************************/
8
9  #ifndef CREATEANDDESTROY_H
10 #define CREATEANDDESTROY_H
11
12 #include <string>
13 using namespace std;
14
15 class CreateAndDestroy
16 {
17 public:
18     /// Constructor
19     CreateAndDestroy(int,string);
20     /// Destructor
21     ~CreateAndDestroy();
22
23 private:
24     int objectID;   // ID number for object
25     string message; // message describing object
26 };
27
28 #endif
```

- The class definition contains prototypes for constructor and destructor.
- It includes private integer member objectID and string message that are used in the program's output to identify the object (lines 21-22).

CreateAndDestroy Class Member Functions

```
1  /* ************************************************************
2   * Filename     :   CreateAndDestroy.cpp
3   * Author       :   Burak Kaleci
4   * Date         :   17.10.2018
5   * Description  :   An example program to introduce
6   *                  constructor and destructor
7   * ************************************************************/
8
9  #include <iostream>
10 #include "CreateAndDestroy.h"
11
12 using namespace std;
13
14 CreateAndDestroy::CreateAndDestroy(int ID ,string msg)
15 {
16     objectID=ID;
17     message=msg;
18
19     cout << "Object " << objectID << "  constructor runs  " << message << endl;
20 }
21
22 CreateAndDestroy::~CreateAndDestroy()
23 {
24     cout << "Object " << objectID << "  destructor runs  " << message << endl;
25 }
```

CreateAndDestroy Class Definition

```
 9  #include <iostream>
10  #include "CreateAndDestroy.h"
11  using namespace std;
12
13  // Function Prototype
14  void create(void);
15
16  // Global Object
17  CreateAndDestroy first(1,"(global before main)");
18
19  int main()
20  {
21      cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
22
23      CreateAndDestroy second( 2, "(local in main)" );
24      static CreateAndDestroy third( 3, "(local static in main)" );
25
26      create(); // call function to create objects
27
28      cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
29      CreateAndDestroy fourth( 4, "(local in main)" );
30      cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
31  }
32
33  void create(void)
34  {
35      cout << "\nCREATE FUNCTION: EXECUTION BEGINS" << endl;
36      CreateAndDestroy fifth( 5, "(local in create)" );
37      static CreateAndDestroy sixth( 6, "(local static in create)" );
38      CreateAndDestroy seventh( 7, "(local in create)" );
39      cout << "\nCREATE FUNCTION: EXECUTION ENDS" << endl;
40  }
```

- Line 17 defines object first in global scope. Its constructor is actually called before any statements in main execute and its destructor is called at program termination after the destructors for all other objects have run.
- Function main declares three objects. Objects second (line 23) and fourth (line 29) are local objects, and object third (line 24) is a static local object.
- The constructor for each of these objects is called when execution reaches the point where that object is declared.
- The destructors for objects fourth then second are called (i.e.,the reverse of the order in which their constructors were called) when execution reaches the end of main.

CreateAndDestroy Class Definition

```
9   #include <iostream>
10  #include "CreateAndDestroy.h"
11  using namespace std;
12
13  // Function Prototype
14  void create(void);
15
16  // Global Object
17  CreateAndDestroy first(1,"(global before main)");
18
19  int main()
20  {
21      cout << "\nMAIN FUNCTION: EXECUTION BEGINS" << endl;
22
23      CreateAndDestroy second( 2, "(local in main)" );
24      static CreateAndDestroy third( 3, "(local static in main)" );
25
26      create(); // call function to create objects
27
28      cout << "\nMAIN FUNCTION: EXECUTION RESUMES" << endl;
29      CreateAndDestroy fourth( 4, "(local in main)" );
30      cout << "\nMAIN FUNCTION: EXECUTION ENDS" << endl;
31  }
32
33  void create(void)
34  {
35      cout << "\nCREATE FUNCTION: EXECUTION BEGINS" << endl;
36      CreateAndDestroy fifth( 5, "(local in create)" );
37      static CreateAndDestroy sixth( 6, "(local static in create)" );
38      CreateAndDestroy seventh( 7, "(local in create)" );
39      cout << "\nCREATE FUNCTION: EXECUTION ENDS" << endl;
40  }
```

- Because object third is static, it exists until program termination.The destructor for object third is called before the destructor for global object first, but after all other objects are destroyed.
- Function create declares three objects: fifth and seventh as local objects and sixth as a static local object.
- The destructors for objects seventh then fifth are called when create terminates. Because sixth is static,it exists until program termination. The destructor for sixth is called before the destructors for third and first,but after all other objects are destroyed.

Introduction

- An lvalue (locator value) represents an object that occupies some identifiable location in memory.
- ```
  int x;
  x=15;
  ```
  In this example, x is a lvalue because it is a variable in integer type and it occupies an identifiable location in memory.
- A reference to an object is an alias for the name of the object and, hence, may be used on the left side of an assignment statement.
- In this context, the reference makes a perfectly acceptable lvalue that can receive a value as given in following statements:
  ```
  int &xRef=x;
  xRef=25;
  ```
  ```

Introduction

- One way to use this capability (unfortunately!) is to have a public member function of a class return a reference to a private data member of that class.
- If a function returns a const reference,that reference cannot be used as a modifiable lvalue.
- Time Class is simplified to demonstrate returning a reference to a private data member with member function badSetHour.
- Such a reference return actually makes a call to memberfunction badSetHour an alias for private data member hour!
- The function call can be used in any way that the private data member canbe used, including as an lvalue in an assignment statement, thus enabling clients of the class to clobber the class's private data at will!
- The same problem would occur if a pointer to the private data were to be returned by the function.

Returning a Reference to a private Data Member

```
1  /* ****************************************************
2   * Filename      :   Time.h
3   * Author        :   Burak Kaleci
4   * Date          :   17.10.2018
5   * Description   :   An example program
6   * ****************************************************/
7
8  // prevent multiple inclusions of header
9  #ifndef TIME_H
10 #define TIME_H
11
12 // Time class definition
13 class Time
14 {
15 public:
16
17     /// Constructor
18     Time(int hr=0, int min=0, int sec=0);
19     /// Set hour, minute and second
20     void setTime(int,int,int);
21     ///
22     int getHour(void);
23     ///
24     int &badSetHour(int); // DANGEROUS reference return
25
26 private:
27
28     int hour; // 0 - 23
29     int minute; // 0 - 59
30     int second; // 0 - 59
31 };
32
33 #endif
```

```
8  #include <stdexcept>    // for invalid_argument exception class
9  #include "Time.h"   // include definition of class Time
10 using namespace std;
11
12 /// Time constructor initializes each data member to zero.
13 Time::Time(int hr, int min, int sec)
14 {
15     setTime(hr,min,sec);
16 }
17
18 /// set new Time value using universal time
19 void Time::setTime(int h ,int m ,int s)
20 {
21     // validate hour, minute and second
22     if ((h>=0 && h<24) && (m>=0 && m<60) && (s>= 0 && s< 60)){
23         hour =h;
24         minute =m;
25         second =s;
26     }
27     else
28         throw invalid_argument("hour,minute and/or second was out of range");
29 }
30
31 int Time::getHour(void)
32 {
33     return hour;
34 }
35
36 // POOR PRACTICE: Returning a reference to a private data member.
37 int &Time::badSetHour(int hh)
38 {
39     if (hh>= 0 && hh < 24 )
40         hour =hh;
41     else
42         throw invalid_argument( "hour mustbe0-23" );
43
44     return hour; // DANGEROUS reference return
45 }
```

Returning a Reference to a private Data Member

```
 8  #include <iostream>
 9  #include <stdexcept>
10  #include "Time.h"   // include definition of class Time
11
12  using namespace std;
13
14  int main() {
15
16      Time t;     // instantiate object t of class Time
17
18      // initialize hourRef with the reference returned by badSetHour
19      int &hourRef =t.badSetHour( 20 ); // 20 is avalid hour
20
21      cout << "Valid hour before modification:"<< hourRef;
22      hourRef =30; // use hourRef to set invalid value in Time object t
23      cout << "\nInvalid hour after modification: " << t.getHour();
24
25      // Dangerous: Function call that returns a reference can be used as an lvalue!
26      t.badSetHour( 12 )=74; // assign another invalid value to hour
27
28      cout << "\n\n**********************************************\n"
29          << "POOR PROGRAMMING PRACTICE!!!!!!!!\n"
30          << "t.badSetHour( 12 ) as an lvalue, invalid hour: "
31          << t.getHour()
32          << "\n**********************************************" << endl;
33
34      return 0;
35  }
```

Line 16 declares Time object t and reference hourRef (line 19), which is initialized with the reference returned by the call t.badSetHour(20).

- Line 21 displays the value of the alias hourRef. This shows how hourRef breaks the encapsulation of the class(statements in main should not have access to the private data of the class).
- Next, line 22 uses the alias to set the value of hour to 30 (an invalid value) and line 23 displays the value returned by function getHour to show that assigning a value to hourRef actually modifies the private data in the Time object t.

Returning a Reference to a private Data Member

```
 8  #include <iostream>
 9  #include <stdexcept>
10  #include "Time.h"   // include definition of class Time
11
12  using namespace std;
13
14  int main() {
15
16      Time t;     // instantiate object t of class Time
17
18      // initialize hourRef with the reference returned by badSetHour
19      int &hourRef =t.badSetHour( 20 ); // 20 is avalid hour
20
21      cout << "Valid hour before modification:"<< hourRef;
22      hourRef =30; // use hourRef to set invalid value in Time object t
23      cout << "\nInvalid hour after modification: " << t.getHour();
24
25      // Dangerous: Function call that returns a reference can be used as an lvalue!
26      t.badSetHour( 12 )=74; // assign another invalid value to hour
27
28      cout << "\n\n**********************************************\n"
29          << "POOR PROGRAMMING PRACTICE!!!!!!!!\n"
30          << "t.badSetHour( 12 ) as an lvalue, invalid hour: "
31          << t.getHour()
32          << "\n**********************************************" << endl;
33
34      return 0;
35  }
```

- Finally,line 26 uses the badSetHour function call itself as an lvalue andassigns 74 (another invalid value) to the reference returned by the function.
- Line 31 again displays the value returned by function getHour to show that assigning a value to the resul tof the function call in line 26 modifies the private data in the Time object t.

Default Memberwise Assignment

- The assignment operator (=) can be used to assign an object to another object of the same type.
- By default, such assignment is performed by memberwise assignment. Each data member of the object on the right of the assignment operator is assigned individually to the same data member in the object on the left of the assignment operator.

Date Class Definition

```
 8  // prevent multiple inclusions of header
 9  #ifndef DATE_H
10  #define DATE_H
11
12  // Date class definition
13  class Date
14  {
15  public:
16
17      /// Default Constructor
18      Date(int m=1, int d=1, int y=2000);
19      void print(void);
20
21  private:
22
23      int month;
24      int day;
25      int year;
26
27  };
28
29  #endif
```

```
 8  #include <iostream>
 9  #include "Date.h"   // include definition of class
10  using namespace std;
11
12  /// Default Constructor
13  Date::Date(int m, int d, int y)
14  {
15      month=m;
16      day=d;
17      year=y;
18  }
19
20  // print Date in the format mm/dd/yyyy
21  void Date::print(void)
22  {
23      cout << month << "/" << day << "/" << year;
24  }
25
```

Driver Program for Date Class

```
 8  #include <iostream>
 9  #include "Date.h"   // include definition of class Date
10  using namespace std;
11
12
13  int main()
14  {
15      Date date1(7,4,2004 );
16      Date date2; // date2 defaults to 1/1/2000
17
18      cout << "date1 =";
19      date1.print();
20      cout << "\ndate2 =";
21      date2.print();
22
23      date2=date1;     // default memberwise assignment
24
25      cout << "\n\nAfter default memberwiseassignment, date2= ";
26      date2.print();
27      cout << endl;
28  }
```

- Line 23 uses default memberwise assignment to assign the data members of Date object date1 to the corresponding data members of Date object date2.
- In this case,the month member of date1 is assigned to the month member of date2,the day member of date1 is assigned to the day member of date2 and the year member of date1 is assigned to the year member of date2.

Introduction

- Some objects need to be modifiable and some do not.
- You may use keyword **const** to specify that an object is not modifiable and that any attempt to modify the object should result in a compilation error.
- The statement

  const Time noon(12,0,0);

  declares a **const** object noon of class Time and initializes it to 12 noon.
- **C++ disallows member function calls for const objects unless the member functions themselves are also declared const.**
- **It means that const objects can only call const member functions.**
- This is true even for get member functions that do not modify the object.

Introduction

- A member function is specified as **const** both in its prototype by inserting the keyword **const** after the function's parameter list and, in the case of the function definition,before the left brace that begins the function body.
- When a **const** member function try to modify a data member of the object it causes a compilation error.
- Invoking a non-const member function on a **const** object is a compilation error.
- A **const** member function can be overloaded with a non-const version. The compiler chooses which overloaded member function to use based on the object on which the function is invoked.
- If the object is **const**,the compiler uses the **const** version. If the object is not const,the compiler uses the non-const version.

Time Class Interface

```
9   #ifndef TIME_H
10  #define TIME_H
11
12  // Time class definition
13  class Time
14  {
15  public:
16
17      /// Default Constructor
18      Time(int hour=0, int minute=0, int second=0);
19      /// Set hour, minute and second
20      void setTime(int,int,int);
21      /// Set hour (after validation)
22      void setHour(int);
23      /// Set minute (after validation)
24      void setMinute(int);
25      /// Set second (after validation)
26      void setSecond(int);
27
28      /// get functions (normally declared const)
29      /// Return hour
30      int getHour() const;
31      /// Return minute
32      int getMinute() const;
33      /// Return second
34      int getSecond() const;
35
36      /// print functions (normally declared const)
37      /// Print time in universal-time format
38      void printUniversal(void) const;
39      /// Print time in standard-time format (should be const)
40      void printStandard(void);
41
42  private:
43
44      int hour; // 0 - 23
45      int minute; // 0 - 59
46      int second; // 0 - 59
47  };
48
49  #endif
```

- In the header Time.h, getHour, getMinute, getSecond and printUniversal member functions are declared as **const**.
- Lines 30, 32, 34 and 38 now include keyword **const** after each function prototype's parameter list.
- Notice that in line 40 printStandard function is not declared as const for pedagogical purposes.
- The corresponding definition of each function in next slayt (lines 55, 61, 67 and 73 , respectively) also specifies keyword const after each function's parameter list.

Time Class Definition

```cpp
#include "Time.h"   // include definition of class Time

using namespace std;

/// Time constructor initializes each data member to zero
Time::Time(int hour, int minute, int second )
{
    setTime( hour, minute, second ); // validate and set time
}
/// Set new Time value using universal time
void Time::setTime(int h, int m, int s)
{
    setHour(h);     // set private field hour
    setMinute(m);   // set private field minute
    setSecond(s);   // set private field second
}
/// Set hour value
void Time::setHour(int h)
{
    if (h >= 0 && h< 24 )
        hour =h;
    else
        throw invalid_argument( "hour must be 0-23" );
}
/// Set minute value
void Time::setMinute(int m)
{
    if (m >= 0 && m< 60 )
        minute =m;
    else
        throw invalid_argument( "minute must be 0-59" );
}
/// Set second value
void Time::setSecond(int s)
{
    if (s >= 0 && s< 60 )
        second =s;
    else
        throw invalid_argument( "second must be 0-59" );
}
```

```cpp
/// Return hour value
int Time::getHour() const // get functions should be const
{
    return hour;
}

/// Return minute value
int Time::getMinute() const
{
    return minute;
}

/// Return second value
int Time::getSecond() const
{
    return second;
}

/// print Time in universal-time format (HH:MM:SS)
void Time::printUniversal(void) const
{
    cout << setfill('0') << setw( 2 ) << getHour() << ":"
        << setw( 2 ) << getMinute() << ":" << setw( 2 ) << getSecond();
}

// print Time in standard-time format (HH:MM:SS AM or PM)
void Time::printStandard(void) // note lack of const declaretion
{
    if (getHour() == 0 || getHour() == 12 )
        cout << 12;
    else
        cout << getHour() % 12;

    cout << ":" << setfill('0') << setw( 2 ) << getMinute() << ":" << setw( 2 )
        << getSecond();

    if (getSecond() < 12)
        cout << " AM";
    else
        cout << " PM";
}
```

Driver Program for Time Class

```
 8  #include <iostream>
 9  #include <stdexcept>
10  #include "Time.h"   // include definition of class Time
11
12  using namespace std;
13
14  int main()
15  {
16      Time wakeUp(6,45,0);         // non-constant object
17      const Time noon(12,0,0);     // const object
18
19
20                                   // OBJECT       MEMBER FUNCTION
21      wakeUp.setHour(18);          // non-const    non-const          valid
22      noon.setHour(12);            // const        non-const          compilation error
23
24      wakeUp.getHour();            // non-const    const              valid
25      noon.getMinute();            // const        const              valid
26
27      noon.printUniversal();       // const        const              valid
28      noon.printStandard();        // const        non-const          compilation error
29
30      return 0;
31  }
```

- The program instantiates two Time objects: non-const object wakeUp (line 16) and const object noon (line 17).
- The program attempts to invoke non-const member functions setHour (line 22)and printStandard (line 28) on the const object noon.
- In each case, the compiler generates an error message.

Driver Program for Time Class

```
 8  #include <iostream>
 9  #include <stdexcept>
10  #include "Time.h"   // include definition of class Time
11
12  using namespace std;
13
14  int main()
15  {
16      Time wakeUp(6,45,0);          // non-constant object
17      const Time noon(12,0,0);      // const object
18
19
20                                    // OBJECT      MEMBER FUNCTION
21      wakeUp.setHour(18);           // non-const   non-const          valid
22      noon.setHour(12);             // const       non-const          compilation error
23
24      wakeUp.getHour();             // non-const   const              valid
25      noon.getMinute();             // const       const              valid
26
27      noon.printUniversal();        // const       const              valid
28      noon.printStandard();         // const       non-const          compilation error
29
30      return 0;
31  }
```

- The program also illustrates the three other member function call combinations on objects: a non-const member function on a non-const object (line 21), a const member function on a non-const object (line 24) and a const member function on a const object(lines 27-28).
- Line 28 generates a compilation error even though member function printStandard of class Time does not modify the object on which it's invoked.
- The fact that a member function does not modify an object is not sufficient to indicate that the function is a constant function. The function must explicitly be declared const.

Constructors with **const** objects

- A constructor must be a **non-const** member function, but it can still be used to initialize a **const** object.
- The Time constructor's definition shows that it calls another non-const member function (setTime) to perform the initialization of a Time object.
- Invoking a non-const member function from the constructor call as part of the initialization of a const object is allowed.
- The"constness" of a const object is enforced from the time the constructor completes initialization of the object until that object's destructor is called.

Initializing a **const** Data Member with a Member Initializer

- All data members can be initialized using member initializer syntax.
- But
  - **const data members**
  - **data members that are references**

  must be initialized using member initializers.

Increment Class Interface

```
 8  // prevent multiple inclusions of header
 9  #ifndef INCREMENT_H
10  #define INCREMENT_H
11
12  // Time class definition
13  class Increment
14  {
15  public:
16      /// Default constructor
17      Increment(int c=0, int i=1);
18      /// Adds increment
19      void addIncrement(void);
20      /// Prints count and increment
21      void print(void) const;
22
23  private:
24      ///
25      int count;
26      /// const data member should be
27      /// initialized with member initializer
28      const int increment;
29  };
30
31  #endif
```

- In the header Increment.h, a default constructor, addIncrement and print member functions are defined.
- Notice that, the print member function is declared as constant.
- Also a non-constant data member count and a const data member increment is defined.

Increment Class Implementation

```
 9  #include <iostream>
10  #include "Increment.h"      // include definition of class Increment
11
12  using namespace std;
13
14  /// Default constructor
15  Increment::Increment(int c, int i)
16      :   count(c),           // initializer for non-const member
17          increment(i)        // required initializer for const member
18  {
19      // empty body
20  }
21
22  /// Adds increment
23  void Increment::addIncrement(void)
24  {
25      count=count+increment;
26  }
27
28  /// Prints count and increment
29  void Increment::print(void) const
30  {
31      cout << "count = " << count << ", increment = " << increment << endl;
32  }
```

- The constructor definition (lines 15-20) uses a member initializer list to initialize class Increment's data members, non-const integer count and const integer increment.
- Member initializers appear between a constructor's parameter list and the left brace that begins the constructor's body.
- The member initializer list is separated from the parameter list with a colon (:).

Increment Class Implementation

```
 9  #include <iostream>
10  #include "Increment.h"      // include definition of class Increment
11
12  using namespace std;
13
14  /// Default constructor
15  Increment::Increment(int c, int i)
16      :   count(c),           // initializer for non-const member
17          increment(i)        // required initializer for const member
18  {
19      // empty body
20  }
21
22  /// Adds increment
23  void Increment::addIncrement(void)
24  {
25      count=count+increment;
26  }
27
28  /// Prints count and increment
29  void Increment::print(void) const
30  {
31      cout << "count = " << count << ", increment = " << increment << endl;
32  }
```

- Each member initializer consists of the data member name followed by parentheses containing the member's initial value.
- In this example, count is initialized with the value of constructor parameter c and increment is initialized with the value of constructor parameter i.
- Multiple member initializers are separated by commas.
- Also,the member initializer list executes before the body of the constructor executes.

Initializing a **const** Data Member with a Member Initializer

```
14  /// Default constructor
15  Increment::Increment(int c, int i)
16  {
17      count=c;
18      increment=i;
19  }
```

Compile Log | Debug | Find Results | Close

	Message
-2019 Fall\Advanced Programming\...	**In constructor 'Increment::Increment(int, int)':**
2019 Fall\Advanced Programming\wee...	[Error] uninitialized const member in 'const int' [-fpermissive]
-2019 Fall\Advanced Programming\...	**In file included from Increment.cpp**
2019 Fall\Advanced Programming\wee...	[Note] 'const int Increment::increment' should be initialized
2019 Fall\Advanced Programming\wee...	[Error] assignment of read-only member 'Increment::increment'
2019 Fall\Advanced Programming\wee...	recipe for target 'Increment.o' failed

- A const object cannot be modified by assignment, so it must be initialized.
- When a data member of a class is declared const,a member initializer must be used to provide the constructor with the initial value of the data member for an object of the class.

Initializing a **const** Data Member with a Member Initializer

```
14  /// Default constructor
15  Increment::Increment(int c, int i)
16  {
17      count=c;
18      increment=i;
19  }
```

Compile Log | Debug | Find Results | Close

	Message
-2019 Fall\Advanced Programming\...	**In constructor 'Increment::Increment(int, int)':**
2019 Fall\Advanced Programming\wee...	[Error] uninitialized const member in 'const int' [-fpermissive]
-2019 Fall\Advanced Programming\...	**In file included from Increment.cpp**
2019 Fall\Advanced Programming\wee...	[Note] 'const int Increment::increment' should be initialized
2019 Fall\Advanced Programming\wee...	[Error] assignment of read-only member 'Increment::increment'
2019 Fall\Advanced Programming\wee...	recipe for target 'Increment.o' failed

- The same is true for references.
- When we attempt to initialize const data member of a class with an assignment statement in the constructor's body rather than with a member initializer, it causes compilation error.
- Notice that a non-const data member like count can be initialized with assignment operator.

Why Is Function print Declared const?

- Function print is declared const. It might seem strange to label this function const, becausea program probably will never have a const Increment object.
- However, it's possible that a program will have a const reference to an Increment object or a pointer to const that points to an Increment object.
- Typically,this occurs when objects of class Increment are passed to functions or returned from functions.
- In these cases, only class Increment's const member functions can be called through the reference or pointer.
- Thus, it's reasonable to declare function print as const. Doing so prevents errors in these situations where an Increment object is treated as a const object.

Introduction

- An AlarmClock object needs to know when it's supposed to sound its alarm, so why not include a Time object as a member of the AlarmClock class?
- Such a capability is called **composition** and is sometimes referred to as a has-a relationship (a class can have objects of other classes as members).
- Previously, we examined how to pass arguments to the constructor of an object we created in main.
- Now we show how an object's constructor can pass arguments to member-object constructors via member initializers.

Date Class Interface

```
 8  // prevent multiple inclusions of header
 9  #ifndef DATE_H
10  #define DATE_H
11
12  // Date class definition
13  class Date
14  {
15  public:
16
17      /// class' data member
18      static const int monthsPerYear=12;
19      /// Default Constructor
20      Date(int m=1, int d=1, int y=2000);
21      /// Print date in month/day/year format
22      void print(void) const;
23      /// Destructor
24      ~Date();
25  private:
26
27      int month;
28      int day;
29      int year;
30
31      /// Utility function to check
32      /// if day is proper for month and year
33      int checkDay(int) const;
34  };
35
36  #endif
```

- In the header Date.h, a default constructor, destructor and print member functions are defined. Also, a private member function which is defined as utility function (checkDay) is used to validate day data member.
- Notice that, the print member function is declared as constant.
- Data members month, day and year is defined in private access identifier. monthsPerYear data member is defined as static const int and is called class' data member.

Date Class Definition

```cpp
#include <iostream>
#include <stdexcept>
#include "Date.h"   // include definition of class Date
using namespace std;

/// Default Constructor
Date::Date(int m, int d, int y)
{
    if (m>0 && m<=monthsPerYear)    // validate the month
        month=m;
    else
        throw invalid_argument("month must be 1-12");

    year=y;             // could validate year
    day=checkDay(d);    // validate the day

    // output Date object to show when its constructor is c
    cout << "Date object constructor for date ";
    print();
    cout << endl;
}

// print Date in the format mm/dd/yyyy
void Date::print(void) const
{
    cout << month << "/" << day << "/" << year;
}
```

```cpp
// output Date object to show when its destructor is calle
Date::~Date()
{
    cout << "Date object destructor for date ";
    print();
    cout << endl;
}
// utility function to confirm proper day value based on
// month and year;handles leap years, too
int Date::checkDay(int testDay) const
{
    static const int daysPerMonth[ monthsPerYear + 1 ]=
        { 0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31 };

    // determine whether testDayisvalidfor specified month
    if (testDay > 0 && testDay <= daysPerMonth[ month])
        return testDay;

    // February 29 check for leap year
    if (month== 2 && testDay == 29 && (year% 400 == 0 ||
        (year % 4 == 0 && year % 100 != 0 )) )
        return testDay;

    throw invalid_argument( "Invalid day for current month and ye
}
```

Employee Class Interface

```
 8  // prevent multiple inclusions of header
 9  #ifndef EMPLOYEE_H
10  #define EMPLOYEE_H
11
12  #include <string>
13  #include "Date.h" // include Date class definition
14  using namespace std;
15
16  // Employee class definition
17  class Employee
18  {
19  public:
20
21      /// Default Constructor
22      Employee(const string &first, const string &last,
23          const Date &dateOfBirth, const Date &dateOfHire);
24      /// print Employee object
25      void print(void) const;
26      /// Destructor
27      ~Employee();
28
29  private:
30
31      ///
32      string firstName;
33      ///
34      string lastName;
35      /// composition: member object
36      const Date birthDate;
37      /// composition: member object
38      const Date hireDate;
39  };
40
41  #endif
```

- Class Employee's definition contains private data members firstName, lastName, birthDate and hireDate.
- Members birthDate and hireDate are const objects of class Date, which contains private data members month, day and year.
- The Employee constructor's header specifies that the constructor has four parameters(first, last, dateOfBirth and dateOfHire).
- The last two are passed via member initializers to the Date class constructor.

Employee Class Definition

```cpp
#include <iostream>
#include "Employee.h"    // include definition of class Employee
#include "Date.h"        // include definition of class Date
using namespace std;

/// constructor uses member initializer list to pass initializer
/// values to constructors of member objects
Employee::Employee(const string &first, const string &last,
        const Date &dateOfBirth, const Date &dateOfHire)
        : firstName(first),         // initialize firstName
          lastName(last),           // initialize lastName
          birthDate(dateOfBirth),   // initialize birthDate
          hireDate(dateOfHire)      // initialize hireDate
{
    // output Employee object to show when constructor is called
    cout << "Employee object constructor:"
        << firstName<< " " << lastName << endl;
}
```

```cpp
/// print Employee object
void Employee::print() const
{
    cout << lastName << ", " << firstName<< " Hired: ";
    hireDate.print();

    cout << " Birthday: ";
    birthDate.print();
    cout << endl;
}

/// output Employee object to show when its destructor is called
Employee::~Employee()
{
    cout << "Employee object destructor:"
        << firstName<< " " << lastName << endl;
}
```

Employee Constructor's Member Initializer List

- The colon (:) following the constructor's header (line 17) begins the member initializer list.
- The member initializers specify the Employee constructor parameters being passed to the constructors of the string and Date data members.
- Parameters first, last, dateOfBirth and dateOfHire are passed to the constructors for objects firstName's (line 17), lastName (line 18), birthDate (line 19) and hireDate (line 20), respectively.
- Again,member initializers are separated by commas.

Driver Program Employee Class

```
 8  #include <iostream>
 9  #include "Employee.h"    // include definition of
10  using namespace std;
11
12  int main()
13  {
14      Date birth(7,24,1949);
15      Date hire(3,12,1988);
16      Employee manager("Bob", "Blue", birth,hire);
17
18      cout << endl;
19      manager.print();
20
21      return 0;
22  }
```

- The program creates two Date objects (lines 14-15) and passes them as arguments to the constructor of the Employee object created in line 16.
- Line 19 outputs the Employee object's data.

- When each Date object is created in lines 14-15, the Date constructor displays a line of output to show that the constructor was called.
- Line 16 causes two additional Date constructor calls that do not appear in the program's output.
- When each of the Employee's Date member object's is initialized in the Employee constructor's member initializer list, the default copy constructor for class Date is called.
- Since this constructor is defined implicitly by the compiler, it does not contain any output statements to demonstrate when it's called.

Driver Program Employee Class

```
Date object constructor for date 7/24/1949
Date object constructor for date 3/12/1988
Employee object constructor:Bob Blue

Blue, Bob Hired: 3/12/1988 Birthday: 7/24/1949
Employee object destructor:Bob Blue
Date object destructor for date 3/12/1988
Date object destructor for date 7/24/1949
Date object destructor for date 3/12/1988
Date object destructor for date 7/24/1949
```

- Class Date and class Employee each include a destructor that prints a message when an object of its class is destructed.
- This enables us to confirm in the program output that objects are constructed from the inside out and destroyed in the reverse order, from the outside in.

- Notice the last four lines in the figure. The last two lines are the outputs of the Date destructor running on Date objects hire (line 15)and birth (line 14), respectively.
- The first two lines are the outputs of the destructors running for the Employee's member objects hireDate and birthDate.
- Again,output figure did not show the constructors running for member objects birthDate and hireDate,because these objects were initialized with the default Date class copy constructors provided by the compiler.

What Happens When I Do Not Use the Member Initializer List?

- If a member object is not initialized through a member initializer, the member object's default constructor will be called implicitly.
- Values,if any, established by the default constructor can be overridden by set functions.
- However, for complex initialization, this approach may require significant additional work and time.
- A compilation error occurs if a member object is not initialized with a member initializer and the member object's class does not provide a default constructor.